

A Case Study of Efficient LongCat-Flash Inference with NVIDIA Dynamo Router: Model-System Co-design for the Agentic Era

A Case Study on High-Performance Inference in LongCat-Flash

Yulei Qian, Fengcun Li

Meituan

GTC 2026

Agenda

- ① Motivation: agentic latency + cost targets
- ② What the model gives the system
- ③ System-level optimization
 - SBO: Single Batch Overlap
 - Multi-step overlapped scheduler
 - PD-disaggregation with layer-wise KV transfer
 - HiCache
 - KV-Router with Nvidia Dynamo
- ④ GPU comm & kernels: NVLink Sharp + SwapAB + Quantization
- ⑤ Performance: measured vs theoretical, gap analysis
- ⑥ Production notes

Motivation

Why agentic inference changes the latency budget

- Agent loops = multiple model calls per task $\Rightarrow$ latency composes
- Two output regimes:
 - **Reasoning text (user-visible):** $\sim$ human reading speed (≈ 20 tok/s)
 - **Action/tool payloads (user-invisible):** 30–100 tokens, needs **as fast as possible**
- Target: $\sim$ **100 tok/s** action decoding $\Rightarrow$ sub-second tool-call startup

Headline targets (what we will engineer for)

- Model scale: **560B MoE**, dynamic activation (avg $\sim 27\text{B}$ active params/token)
- Efficiency: **>100 tok/s** on Hopper GPU in large-scale deployment
- Cost point: **\$0.70 per 1M output tokens** under \$2/Hopper GPU-hour assumption
- Goal: balance **throughput** and **single-user latency** without violating SLA

Metrics contract

- **TPS/u**: tokens/sec per user (interactive speed)
- **TGS**: tokens/GPU/sec (cost efficiency proxy)
- **TTFT**: time to first token
- **TPOT**: time per output token

Design principle

Optimize end-to-end **agent loop latency** under real traffic while keeping **TGS** high.

Where the bottlenecks come from (decoding)

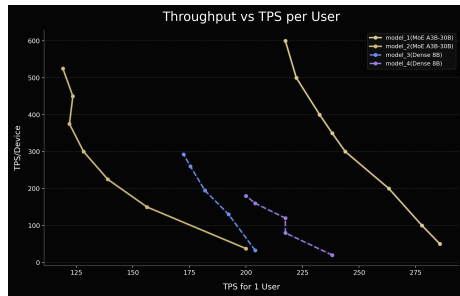
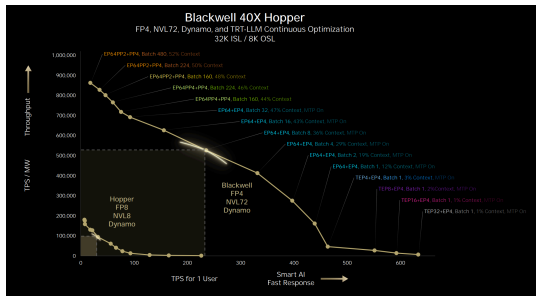
- MoE introduces heavy communication: dispatch / combine
- Attention introduces KV I/O bandwidth pressure
- Decoding often becomes **launch-bound** (many small kernels)
- PD-disaggregation adds KV transfer cost across nodes

Key idea

Create and exploit **overlap windows** so NVLink, NIC, SMs, and Memory access run concurrently.

What the Model Gives the System

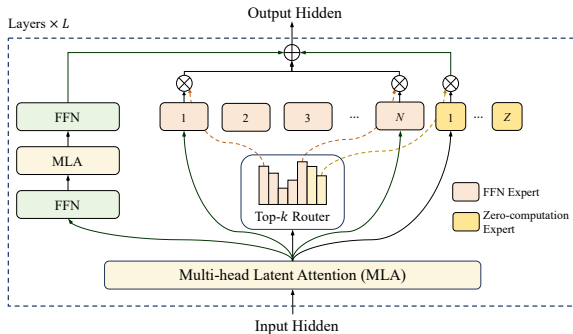
Why model-system co-design?



Architecture features we actually care about (systems lens)

- **ScMoE (Shortcut-connected MoE):** enlarges compute–comm overlap window
- **Zero-computation experts:** dynamic compute budget; some routes incur no extra compute
- **N-gram embedding:** scaling embeddings outperforms scaling experts
- **MLA KV compression:** reduces KV storage/bandwidth pressure
- **Lightweight MTP:** enables speculative decoding with high acceptance

ScMoE: overlap as a structural property



- Conventional MoE: dispatch $\rightarrow$ compute $\rightarrow$ combine (often serialized)
- **ScMoE**: reorders pipeline so compute overlaps dispatch/combine

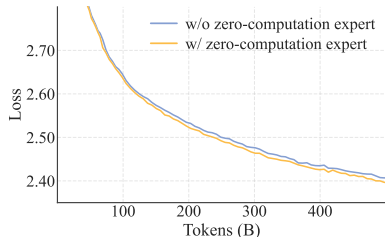
Zero-computation experts: dynamic compute without extra work

- Expand expert pool with zero-computation experts (identity)
- Router still selects top- k experts, but **effective compute varies per token**
- Active params/token: **18.6B–31.3B** (avg $\sim 27\text{B}$)

Systems implication

Some routed mass can become **no-comm** / **no-extra-compute** fast path (with proper runtime support).

Zero-computation experts(Cont.)



Layer 1 - Math

Below is an instruction that describes a task. Write a response that appropriately completes the request.

Question: A robe takes 2 bolts of blue fiber and half that much white fiber. How many bolts in total does it take?

Let's think step by step.

Answer: It takes $2/2 = 1$ bolt of white fiber. So the total amount of fabric is $2 + 1 = 3$ bolts of fabric.

The answer is 3.

Question: In a dance class of 20 students, 20% enrolled in contemporary dance, 25% of the remaining enrolled in jazz dance, and the rest enrolled in hip-hop dance. What percentage of the entire students enrolled in hip-hop dance?

Let's think step by step.

Layer 1 - Code

```
def quick_sort(arr: list) -> list:
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

if __name__ == "__main__":
    unsorted_list = [3, 6, 8, 10, 1, 2, 1, 5, 7, 4]
    sorted_list = quick_sort(unsorted_list)
    print(sorted_list)
```

Layer 1 - Chinese

宇宙中通常所说的“大爆炸”是指：宇宙是在过去有限的时间之前，由一个密度极大且温度极高的太初状态（Initial singularity）演变而来的。根据2015年普朗克卫星所得到的最佳观测结果，宇宙大爆炸距今 137.99 ± 0.21 亿年，并经过不断的膨胀到达今天的状态。大爆炸这一模型的框架基于爱因斯坦的广义相对论，又在场方程求解上作出了一定的简化（例如宇宙学原理假设空间的均匀性和各向同性）。1922年，苏联物理学家亚历山大·弗里德曼用广义相对论描述了流体，从而给出了这一模型的场方程。1927年，比利时物理学家乔治·勒梅特通过求解弗里德曼方程已经在理论上提出了同样的观点。这个解后来被称为弗里德曼-勒梅特-罗伯逊-沃尔克度规。

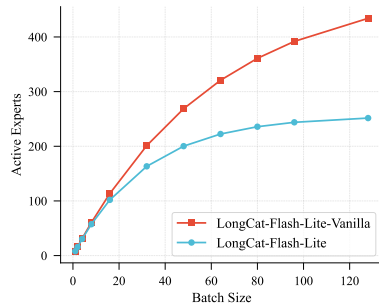
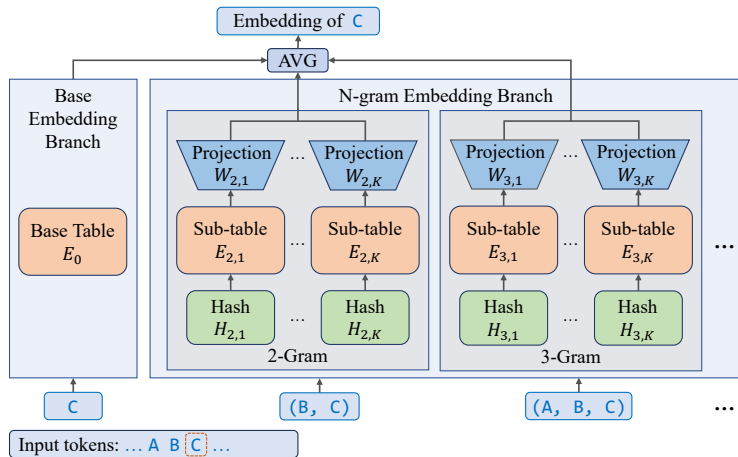


Table: The number of activated FFN experts per token across layers.

N-gram embedding

- In specific sparsity regimes, **N-gram embedding scaling** beats adding experts on the quality-efficiency Pareto frontier
- Key rules from experiments:
 - introduce N-gram embeddings after expert scaling passes its “sweet spot”;
 - keep N-gram embedding budget $\leq$ **50%** of total parameters,
 - avoid vocab sizes near integer multiples of base vocab (reduce hash collisions),
 - wider models benefit more, while gains diminish with excessive depth.
- Systems implication: shift parameters from active MoE compute to embeddings to lower decode-time active compute/I/O; combine with N-gram cache + speculative decoding for real speedups

N-gram embedding(Cont.)



KV pressure: MLA as a systems enabler

- MLA reduces attention compute load while enabling **strong KV cache compression**
- Less KV storage/bandwidth pressure $\Rightarrow$ easier overlap and disaggregated transfer

Practical framing

If KV is cheaper, you can:

- increase concurrency before HBM becomes bottleneck,
- reduce TTFT under PD-disaggregation.

Speculative decoding enabler: lightweight MTP

- MTP as draft model for speculative decoding
- Single dense layer MTP head can reach **>90% acceptance**
- But speculation can amplify **scheduling overhead** unless fused/pipelined

System-Level Optimization

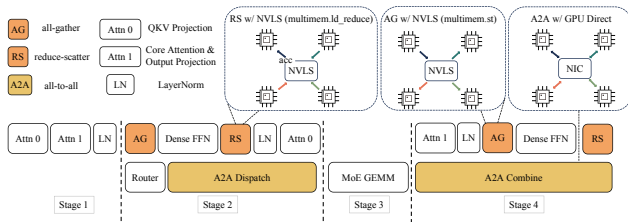
Inference co-design problem statement

- Two core challenges:
 - ① Computation–communication orchestration
 - ② KV cache I/O and storage
- Prior overlap levels: operator / expert / layer
- ScMoE introduces **module-level overlap** $\Rightarrow$ new scheduling space

SBO: Single Batch Overlap

- SBO is a **four-stage pipeline** that hides comm inside **one batch**
- Contrasts with Two-Batch Overlap (TBO): hide comm across batches
- Goal: maximize overlap while keeping **TPS/u** high and latency low

SBO: Stage Breakdown



- **Stage 1**: MLA output feeds later stages $\Rightarrow$ run separately
- **Stage 2**: overlap **A2A dispatch** with **Dense FFN + Attn0 (QKV proj)**
- **Stage 3**: MoE GEMM standalone; benefits from wide EP
- **Stage 4**: overlap **A2A combine** with **Attn1 (core attn + out proj) + Dense FFN**

SBO: Heterogeneous overlap, NVLink + RDMA in the same pipeline

- ScMoE + SBO expose windows to overlap **NVLink utilization** with **NIC utilization**
 - Intra-node TP comm via **NVLink**
 - Inter-node EP comm via **RDMA** (GPUDirect RDMA)

Practical goal

Keep **NVLink**, **NIC**, **SMs**, **Memory access** busy concurrently.

Launch-bound decoding: why scheduling becomes the bottleneck

- Decoding: small/irregular batches + many kernels $\Rightarrow$ kernel launch overhead dominates
- Speculative decoding adds even more scheduling (target + verify + draft)

Fix: TVD fusing

Fuse **Target forward** + **Verification** + **Draft forward** into a single **CUDA graph**.

Multi-step overlapped scheduler

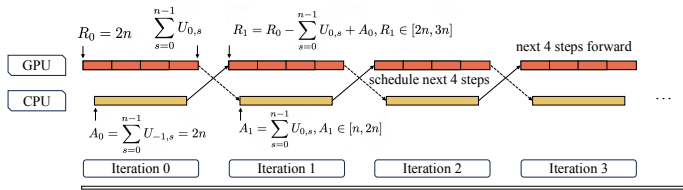
- Single-step pre-schedule is insufficient when forward pass is very low-latency
- Multi-step overlapped scheduler launches kernels for **multiple future steps per iteration**
- Hides CPU scheduling/synchronization **under GPU forward**

The tricky part: KV pre-allocation under speculation

- Multi-step scheduler must pre-allocate KV slots for future steps
- But accept length from speculative decoding is unknown at allocation time
- Need to guarantee allocation does not diverge (stability + bounded growth)

Invariant:

For MTP=1 and n -step scheduling, available entries R_i remains bounded ($R_i \in [2n, 3n]$).



KV pre-allocation (math sketch)

Given n -step multi-step scheduling and $\text{MTP}=1$:

$$R_0 = 2n, \quad A_i = \sum_{s=0}^{n-1} U_{i-1,s}, \quad R_i = R_{i-1} - \sum_{s=0}^{n-1} U_{i-1,s} + A_{i-1}$$

Closed form (by induction):

$$R_i = 4n - \sum_{s=0}^{n-1} U_{i-1,s} \Rightarrow R_i \in [2n, 3n].$$

PD-disaggregation: decouple prefill and decode

- Prefill is compute-bound; decode is often memory/launch-bound
- PD-disaggregation allows independent scaling/optimization
- New bottleneck: **KV cache transfer** (prefill → decode)

Layer-wise KV transmission:

- Key idea: **stream KV layer-by-layer** instead of waiting for full KV blob
- Return the first token sooner
- Starts decoding earlier

Why HiCache Now

Problem shift in the agentic era

Device memory is finite, but agentic inference creates much larger and longer-lived KV footprints (longer contexts, more turns, and higher concurrency).

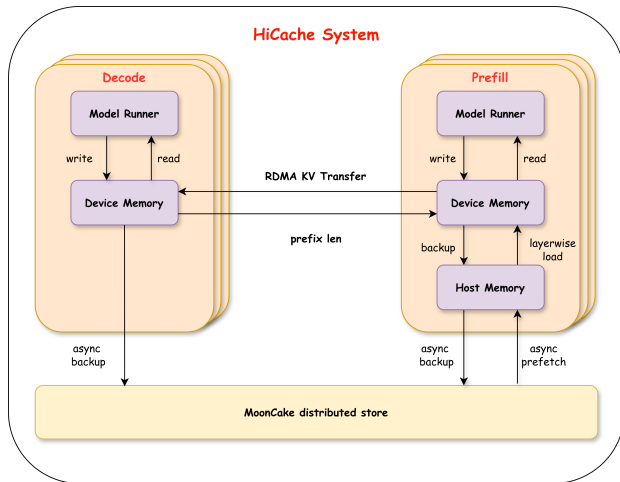
What the system now requires

- **Capacity scaling:** extend KV beyond a single GPU across HBM/DRAM/SSD tiers.
- **Reuse efficiency:** preserve prefix reuse to avoid redundant KV replicas.
- **Latency control:** move KV proactively so offloading does not slow decoding.

HiCache positioning

HiCache is the cache infrastructure layer for multi-tier KV offloading and cluster-wide global sharing.

How HiCache Delivers



- **Incremental KV transfer:** decode-tree cache sends only KV deltas from prefill.
- **Global KV pool:** multiple PD nodes share one distributed cache pool.
- **Compute-I/O overlap:** async read/write and optimized kernels hide KV transfer latency.

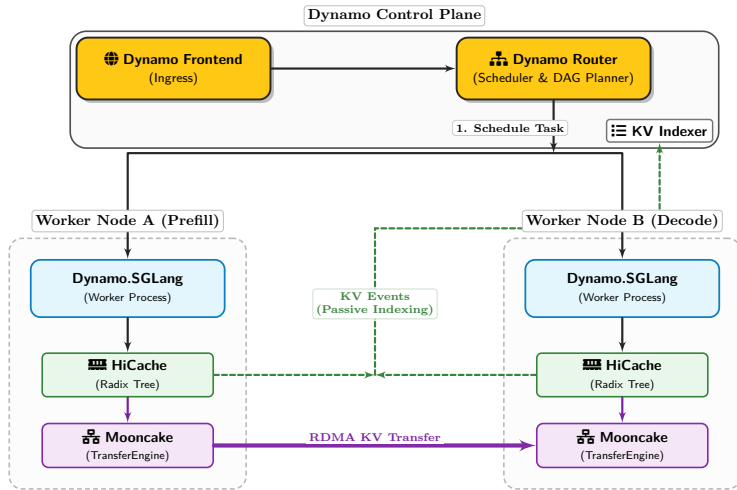
KV-Router with Nvidia Dynamo

- **Keep Dynamo's "brain" in the control plane:**
 - KV-aware routing with a global Radix-Tree index (Crucial for multi-turn agent loops)
 - Scheduling for prefill/decode disaggregation
- **Re-engineer the "body" for LongCat-Flash execution:**
 - Customized SGLang workers with native HiCache
 - Mooncake-based RDMA transport for zero-copy cross-node KV transfer
- **Event-driven passive indexing closes the loop:**
 - HiCache publishes KV events from workers
 - KV Router maintains a shadow index and routes to cache-hot workers

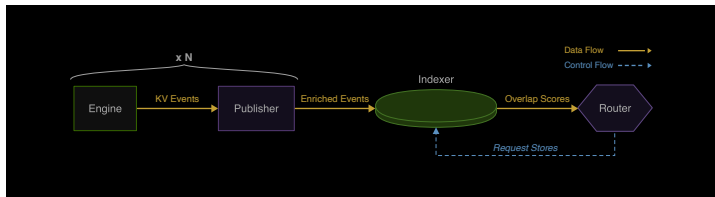
Impact on Agentic Inference

Maximizes KV cache reuse across multi-step reasoning. Compared to random routing, KV-aware routing achieves **~20% higher cache hit rate** and **2.0X faster P50 TTFT**, enabling sub-second tool-call startup.

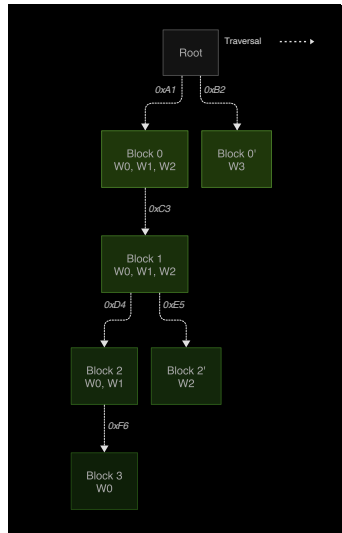
KV-Router with Nvidia Dynamo (Architecture)



Radix Tree

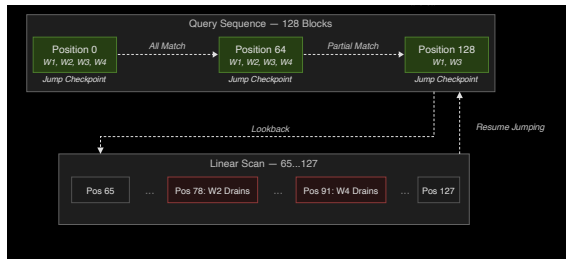


```
type SharedRadixBlock = Rc<RefCell<RadixBlock>>;  
struct RadixBlock {  
    children: HashMap<LocalHash, SharedRadixBlock>,  
    workers: HashSet<WorkerWithDpRank>,  
    block_hash: Option<ExternalHash>,  
}  
struct RadixTree {  
    root: SharedRadixBlock,  
    lookup: HashMap<WorkerWithDpRank, HashMap<ExternalHash, SharedRadixBlock>>,  
}
```



Positional Indexer with Jump Search

```
enum SeqEntry {  
    Single(ExternalHash, HashSet<WorkerWithDpRank>),  
    Multi(HashMap<ExternalHash, HashSet<WorkerWithDpRank>>),  
}  
  
struct PositionalIndexer {  
    // index[position] -> { local_hash -> SeqEntry }  
    index: Vec<DashMap<LocalHash, SeqEntry>>,  
    worker_blocks: DashMap<WorkerWithDpRank, LevelIndex>,  
    jump_size: usize,  
}
```



GPU Comm & Kernels: NVLink Sharp + SwapAB +
Quantization

NVLink Sharp as a first-class primitive

- Goal: minimize data movement and SM occupancy for collectives
- Use NVLink Sharp hardware features:
 - **multimem.st** (hardware broadcast)
 - **multimem.ld_reduce** (in-switch reduction)
- Inline PTX reduce-scatter / all-gather kernels
- Support uniform + non-uniform token distributions
- Reported: outperform NCCL and MSCCL++ across 4KB–96MB using only 4 thread blocks

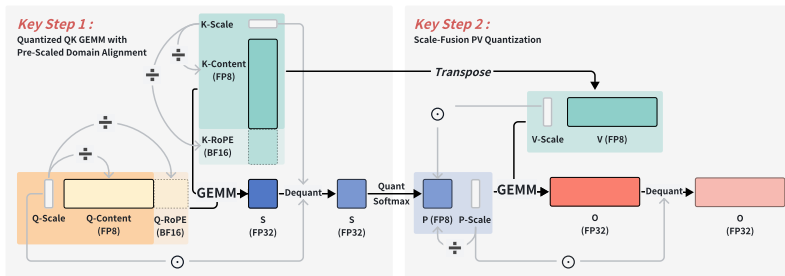
Engineering message

Communication efficiency is not just bandwidth; it is **launch overhead + SM occupancy + pattern matching**.

GEMM kernel: SwapAB to avoid padding

- Decoding often memory-bound with small, irregular token counts
- Conventional mapping requires padding when token count m is below minimum tile
- **SwapAB**: treat weights as LHS and activations as RHS
- Exploit flexible n -dimension granularity (8 elements) to maximize tensor core utilization

Quantization: SnapMLA

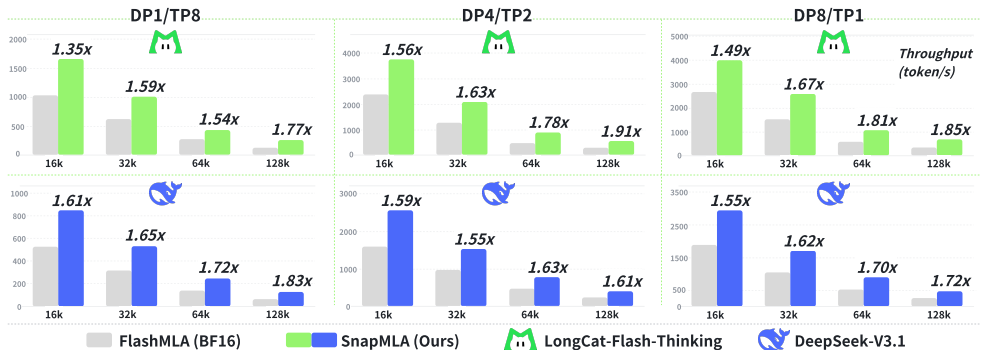


SnapMLA Scale fusion overview

- **Key Step 1: RoPE-aware per-token KV quantization:** Pre-scale BF16 RoPE part of QK with content quantization scale to unify numeric domains.
- **Key Step 2: Quantized PV computation pipeline reconstruction:** Pre-fuse the scale of V into P to avoid quantization-dimension mismatch.

Note: K and V content share one latent cache (c_{KV}).

Quantization: SnapMLA(Cont.)



End-to-end decoding throughput comparison

We evaluate SnapMLA generation throughput on **DeepSeek-R1** and **LongCat-Flash-Thinking** across different parallelization configurations and context lengths.

Performance: Measured vs Theoretical, Gap Analysis

Theoretical performance

	DeepSeek-V3	Qwen3-235B-A22B	LongCat-Flash
MTP	w/	w/o	w/
# layers	61	94	28
batch/device	96	96	96
attention (μs)	471	314	264
A2A dispatch (μs)	275	157	236
MoE (μs)	77	29	60
A2A combine (μs)	551	315	472
overlap strategy	TBO	TBO	SBO
TPOT (ms)	30	26.2	16
\$/1M output token	0.17	0.15	0.09

Key message

SBO + reduced layer count drives down end-to-end latency even if per-layer time is similar.

Measured performance

Model	Prec.	Avg ctx	#GPUs	TGS	TPS/u
DeepSeek-V3-profile	bf16	4096	128	2324	20
DeepSeek-V3-blog	bf16	4989	144	1850	20–22
LongCat-Flash	bf16	5000	128	3785	35
LongCat-Flash	bf16	5000	128	2205	68.9
LongCat-Flash	bf16	5000	128	804	100.5
LongCat-Flash	fp8	5000	128	4230	26.4
LongCat-Flash	fp8	8192	128	3240	33.8

Interpretation

Multiple operating points: you can trade **TGS** for **TPS/u** depending on product goals.

From theory to practice: where the gap comes from

- Reported example (batch/device=96):
 - theoretical **TPOT** ≈ 16 ms (SBO)
 - measured **TPOT** ≈ 26 ms
- Main sources of gap:
 - **small-operator overhead**
 - **communication bandwidth loss**

Engineering implication

After overlap is “good enough”, the last mile is **kernel fusion + comm efficiency + launch reduction**.

Production Notes

- **Model structure buys overlap budget:** ScMoE + zero-experts + MLA + MTP
- **Scheduler converts overlap to performance:** SBO + multi-step + CUDA graphs
- **System-level optimization delivers UX:** PD-disaggregation + layer-wise KV + kernels
- Result: high **TPS/u** (agentic) at competitive **TGS** (cost), without breaking SLA